

QUANTUM PORTFOLIO OPTIMIZER

Overview

The "quantum computing" feature in this project is a quantum-inspired portfolio optimizer, not a true quantum hardware implementation. There is no Qiskit, no quantum circuit execution, and no qubit-based runtime in this repository. Instead, the system uses a classical optimization service written in Java that evaluates holdings, estimates return and risk, and suggests portfolio rebalancing actions.

The backend exposes this functionality through a Spring endpoint and the frontend calls it from the Holdings page.

Main Backend Files

- 'demo/src/main/java/legacy/firstmodel/controller/QuantumOptimizationController.java'
- 'demo/src/main/java/legacy/firstmodel/service/QuantumOptimizationService.java'
- 'demo/src/main/java/legacy/firstmodel/dto/OptimizationRequestDTO.java'
- 'demo/src/main/java/legacy/firstmodel/dto/OptimizationResponseDTO.java'

Main Frontend Files

- 'legacy-portfolio-app/src/services/quantumService.js'
- 'legacy-portfolio-app/src/Holdings.jsx'
- 'legacy-portfolio-app/src/components/QuantumOptimizerCard.jsx'
- 'legacy-portfolio-app/src/components/OptimizationResultsPanel.jsx'
- 'legacy-portfolio-app/src/components/AllocationComparisonChart.jsx'

End-to-End Flow

1. The user opens the Holdings page and expands the Quantum Optimizer.
2. The frontend form collects:
 - risk tolerance
 - target objective
 - max holdings
 - cash available
3. The frontend sends a 'POST /quantum/optimize' request.
4. The backend controller forwards the request to 'QuantumOptimizationService'.
5. The service evaluates current holdings, estimates return and risk, computes target weights, and returns recommendations.
6. The frontend renders optimization metrics, allocation comparisons, and buy/sell/hold suggestions.

What 'QuantumOptimizationService' Does

'optimize(...)'

This is the main orchestration method. It:

1. Loads the current holdings.
2. Validates that holdings exist.
3. Reads user preferences from the request.
4. Calculates current portfolio value.
5. Adds available cash to determine investable value.
6. Builds asset-level metrics.
7. Selects the best assets based on score.
8. Computes target allocation weights.
9. Builds:
 - current allocations
 - optimized allocations
 - buy/sell/hold recommendations
10. Calculates:
 - expected return
 - expected risk
 - Sharpe ratio
11. Returns the final response DTO.

'buildAssetMetrics(...)'

This method creates a scoreable record for each holding. For each asset it calculates:

- current value
- current portfolio weight
- expected return
- risk score
- final optimization score

The core scoring idea is:

'score = adjusted return - (lambda x adjusted risk)'

So:

- higher return improves the score
- higher risk lowers the score
- lambda controls how strongly risk is penalized

'estimateRiskReturn(...)'

This method estimates expected return and expected risk for each asset.

It:

- fetches price history using 'priceService.getHistory(symbol)'
- extracts close prices
- calculates historical return
- calculates daily volatility
- annualizes volatility
- blends historical return with the holding's existing profit percentage

If historical data is unavailable, the code falls back to simpler holding-level metrics.

'lambdaForRiskTolerance(...)'

This maps the user's risk setting into a risk penalty:

- low risk -> stronger penalty on risk
- medium risk -> balanced penalty
- high risk -> lighter penalty on risk

This is how the optimizer adapts to different investor preferences.

'adjustReturnForObjective(...)' and 'adjustRiskForObjective(...)'

These methods modify return and risk scores depending on the selected investment objective:

- growth
- balanced
- low-risk

For example, a growth objective boosts return emphasis, while a low-risk objective penalizes risk more heavily.

'computeTargetWeights(...)'

This is where the recommended portfolio allocation is produced.

The method:

1. Ranks assets by score.
2. Keeps the top N assets based on 'maxHoldings'.
3. Shifts scores into positive space.
4. Normalizes them into target weights that sum to 100%.

If scores are too flat, it falls back to equal weighting.

'buildRecommendations(...)'

This compares current allocation weight against target allocation weight and assigns:

- 'BUY' if target weight is meaningfully above current weight
- 'SELL' if target weight is meaningfully below current weight
- 'HOLD' if the difference is small

These actions are what appear in the frontend recommendations table.

'weightedExpectedReturn(...)', 'weightedExpectedRisk(...)', and 'calculateSharpeRatio(...)'

These methods produce portfolio-level summary metrics:

- expected return
- expected risk
- Sharpe ratio

The Sharpe ratio gives a simple measure of return relative to risk.

Why It Is Called Quantum-Inspired

It is called quantum-inspired because it frames the problem as an optimization task and ranks assets under constraints, similar to how advanced optimization systems are often presented in finance and quantum research discussions.

However, this implementation is still classical. It does not:

- create qubits
- execute quantum circuits
- call IBM Qiskit, D-Wave, Cirq, or PennyLane
- run on quantum hardware

So the current version is best described as a quantum-inspired optimization demo.

Impact of This Feature

This feature adds value because it:

- demonstrates advanced portfolio optimization concepts
- gives users interpretable buy/sell/hold guidance
- creates a strong technical showcase for the project

- keeps a clean API contract that could later support a more advanced optimizer

Limitations

The current implementation has important limitations:

- it is not true quantum computing
- it uses heuristic scoring rather than a mathematically exact global optimizer
- it depends on market history availability
- it performs per-symbol history fetching, which can increase latency and API usage
- recommendation quality depends on the quality of fallback data

What Would Happen at Scale

If this exact implementation were deployed at scale, the main issues would be:

1. Higher latency

The optimizer fetches historical data for each symbol. Larger portfolios and more users would increase response time.

2. API quota pressure

Repeated history lookups across many users would put pressure on external market-data APIs.

3. More backend load

All optimization work currently runs synchronously in the request path. High traffic would increase CPU and thread pressure.

4. Inconsistent result quality

When historical data is unavailable, the service falls back to simpler metrics. That means recommendation quality can vary.

5. No real quantum advantage

Scaling this service would only scale a classical heuristic optimizer, not a true quantum engine.

What Would Be Needed for Production Scale

To support large-scale deployment, the system would likely need:

- cached and preprocessed historical market data

- batched or asynchronous optimization jobs
- stronger portfolio risk modeling, including covariance-aware optimization
- better monitoring, quotas, and retry handling
- clearer auditability for recommendations
- a pluggable optimizer architecture for swapping in a true external optimization engine later

Best Summary

This project contains a well-structured quantum-inspired portfolio optimization feature built with:

- a React frontend form and results UI
- a Spring controller and DTO contract
- a Java optimization service
- allocation and recommendation visualizations

It is strong as an academic or demo feature, but if implemented at scale in its current form, the main bottlenecks would come from market-data lookups, synchronous backend compute, and heuristic model quality rather than frontend design or API shape.